TREINAMENTO CYPRESS

APOSTILA DO ALUNO · DIA 3

Treinamento Cypress

DIA 01
Fundamentos

DIA 02
Construindo uma suíte

DIA 03
Profissionalizando

```
$ npx cypress run

  Página de login
    ✓ deve carregar a tela de login
    ✓ deve permitir login com credenciais válidas de administrador
    ✓ deve exibir mensagem de erro ao informar senha inválida

  Cadastro de usuário
    ✓ deve cadastrar um usuário comum

  76 passing
```

CONTEÚDO

Sumário

03 DIA 3 — **Profissionalizando** 3

3.1 Retomada 3
3.2 Page Objects e arquitetura 3
3.3 Boas práticas e antipadrões 8
3.4 Cucumber (visão geral) 10
3.5 CI/CD com Docker e Jenkins 13
3.6 Relatórios 19
3.7 Banco de dados com cy.task 21
3.8 Lab final, code review e encerramento 29
As oito regras 35
Para onde ir depois 36

APÊNDICES — **Consulta rápida** 37

A Comandos do Cypress por finalidade 37
B Tabela de asserções 39
C Como ler uma falha: o método de 4 passos 41
D Mensagens de erro e o que significam 42
E Seletores do ServeRest 43
F Comandos customizados do projeto 46
G Endpoints da API do ServeRest 47
H As oito regras 49
I Para onde ir depois 50

03

DIA 3

Profissionalizando

AO FIM DESTE DIA VOCÊ TERÁ

um projeto que outra pessoa consegue manter, critério para reconhecer teste ruim, a suíte rodando fora da sua máquina, e o caminho para conferir dados direto no banco.

3.1 Retomada

Hoje o assunto muda de natureza. Nos dois primeiros dias a pergunta era *"como faço isso funcionar?"*. Hoje é **"como faço isso durar?"**.

É a diferença entre um teste que funciona na sua máquina e um teste que funciona no time, seis meses depois, com outra pessoa mexendo nele.

3.2 Page Objects e arquitetura

Respondendo a pergunta de ontem

Se o `data-testid` do botão entrar mudar, quantos arquivos você precisa abrir?

Provavelmente uns oito. Vamos transformar isso em um.

Passo 1 — centralizar os seletores

O primeiro passo é quase bobo de tão simples. Um arquivo, `cypress/support/selectors.js`:

JAVASCRIPT

```
export const login = {
  email: '[data-testid="email"]',
  senha: '[data-testid="senha"]',
  entrar: '[data-testid="entrar"]',
  alerta: '.alert'
}
```

E nos testes:

```javascript
import { login } from '../../support/selectors'

cy.get(login.email).type('admin.treinamento@qa.com')
```

Pronto. Agora é **um** arquivo. Esse é o argumento inteiro, e ele se explica sozinho.

Passo 2 — o Page Object

Centralizar seletor resolve metade do problema. A outra metade é que o *jeito de usar a tela* também está espalhado: a sequência de preencher e-mail, senha e clicar está repetida em todo lugar.

Um Page Object é um objeto que sabe tudo sobre **uma tela**.

Um mínimo de classes, só o que você precisa saber:

```javascript
class LoginPage {
  // propriedade: um dado que pertence ao objeto
  seletores = { email: '[data-testid="email"]' }

  // método: uma ação que o objeto sabe fazer
  visitar() {
    cy.visit('/login')
    return this        // devolve o próprio objeto — já já a gente volta nisso
  }
}

export default new LoginPage()   // exporta um objeto pronto, não a classe
```

Três palavras novas:

- `class` — o molde
- `this` — "eu mesmo", o objeto
- `new` — cria um objeto a partir do molde

Não vamos usar herança nem nada além disso.

O Page Object completo (`cypress/pages/LoginPage.js`):

JAVASCRIPT

```javascript
import { login } from '../support/selectors'

class LoginPage {
  seletores = login

  visitar() {
    cy.visit('/login')
    return this
  }

  preencherEmail(email) {
    if (email) cy.get(this.seletores.email).type(email)
    return this
  }

  preencherSenha(senha) {
    if (senha) cy.get(this.seletores.senha).type(senha)
    return this
  }

  clicarEntrar() {
    cy.get(this.seletores.entrar).click()
    return this
  }

  logar(email, senha) {
    this.visitar()
    this.preencherEmail(email)
    this.preencherSenha(senha)
    this.clicarEntrar()
    return this
  }

  validarMensagemErro(mensagem) {
    // 15 s: o alerta de login às vezes demora mais que os 10 s padrão do projeto.
    cy.get(this.seletores.alerta, { timeout: 15000 }).should('be.visible').and('contain', mensagem)
    return this
  }

  validarRedirecionamentoAdmin() {
    cy.url().should('include', '/admin/home')
    return this
  }
}

export default new LoginPage()
```

O `return this` e por que ele importa

Todo método devolve `this` — o próprio objeto. É isso que permite encadear:

```javascript
import LoginPage from '../../pages/LoginPage'

it('deve exibir erro com senha inválida', () => {
  LoginPage
    .logar('admin.treinamento@qa.com', 'senhaerrada')
    .validarMensagemErro('Email e/ou senha inválidos')
})
```

Leia o teste em voz alta. Ele virou uma frase em português. Compare com a versão de ontem:

```javascript
it('deve exibir erro com senha inválida', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('senhaerrada')
  cy.get('[data-testid="entrar"]').click()
  cy.get('.alert').should('contain', 'Email e/ou senha inválidos')
})
```

A segunda versão não é errada. Mas ela descreve **como clicar**; a primeira descreve **o que está sendo testado**. Quando alguém abrir sua suíte daqui a um ano, a diferença é enorme.

> **DICA**
>
> **Sobre o `if` dentro de `preencherEmail`.** Daqui a pouco, no bloco 3.3, você vai ver que `if` decidindo o que validar é antipadrão. Repare que aqui ele está no Page Object, não no teste — e serve para uma situação técnica (o `cy.type()` recusa string vazia, e a gente precisa do cenário "campo em branco"). A diferença é que ele não decide **o que validar**; só ajusta **como preencher**.

Quando vale e quando é exagero

Vale quando:

- o projeto tem mais de uma dezena de arquivos de teste
- mais de uma pessoa mexe nele
- a aplicação muda com frequência

É exagero quando:

- são cinco testes
- você está sozinho
- a tela não muda há um ano

DICA

Page Object tem custo. Mais arquivos, mais indireção, mais um lugar para procurar quando algo quebra. Aplicar cedo demais em projeto pequeno é burocracia sem retorno. Ninguém vai te elogiar por ter uma arquitetura linda numa suíte de três testes.

Comando customizado ou Page Object?

	Para quê	Exemplo
Comando customizado	Ação transversal, serve em qualquer tela	`cy.loginViaApi()`, `cy.criarUsuarioViaApi()`
Page Object	Conhecimento sobre **uma** tela específica	`LoginPage`, `ProdutosPage`

Os dois convivem. Não é escolha excludente. Um teste típico do projeto usa os dois juntos:

JAVASCRIPT

```
beforeEach(() => {
  cy.loginViaApi()                  // comando: preparar estado
})

it('deve cadastrar um produto pela interface', () => {
  ProdutosPage.cadastrar(produto)   // page object: usar a tela
  ProdutosPage.visitarListagem().validarProdutoNaListagem(produto.nome)
})
```

Para praticar em casa: com o projeto instalado (bloco 1.3), pegue o lab do Dia 2 e converta para Page Object. Coloque as duas versões lado a lado no final.

3.3 Boas práticas e antipadrões

O catálogo

#	Antipadrão	Por que quebra
1	`cy.wait(3000)`	Lento quando funciona, instável quando não
2	Teste dependente de ordem	Quebra em paralelo e no CI
3	Seletor por classe CSS ou caminho estrutural	Quebra na primeira mudança de layout
4	Massa fixa em ambiente compartilhado	Quebra quando outra pessoa usa o mesmo dado
5	Login pela tela em todo teste	Multiplica tempo e pontos de falha
6	Teste sem asserção	Passa sempre, não protege nada
7	Um teste gigante com trinta passos	Quando falha, não se sabe onde
8	Nome vago (`it('teste 1')`)	Relatório inútil
9	`if` decidindo o que validar	Teste que se adapta ao resultado não valida nada
10	Comentar o teste que falha	Dívida silenciosa: ou conserta, ou remove

O antipadrão 6, com nome e sobrenome

JAVASCRIPT

```javascript
// ❌ RUIM — não valida nada
it('deve fazer login', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('treino123')
  cy.get('[data-testid="entrar"]').click()
})
```

Esse teste **passa mesmo se o login estiver completamente quebrado**. Ele clica no botão e termina. Se a aplicação responder com erro 500, ele passa. Se a tela não redirecionar, ele passa.

JAVASCRIPT

```javascript
// ✅ BOM
it('deve redirecionar para a home do admin após login válido', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('treino123')
  cy.get('[data-testid="entrar"]').click()

  cy.url().should('include', '/admin/home')
  cy.contains('h1', 'Bem Vindo').should('be.visible')
  cy.get('[data-testid="logout"]').should('be.visible')
})
```

DICA

Se esse exemplo te deixou desconfortável, você está em boa companhia. Praticamente todo mundo já escreveu o primeiro — inclusive quem está te ensinando.

O antipadrão 9, que é mais sutil

JAVASCRIPT

```javascript
// ❌ RUIM — o teste se adapta e nunca falha
cy.get('body').then(($body) => {
  if ($body.find('tbody tr').length > 0) {
    cy.get('tbody tr').should('exist')
  } else {
    cy.log('sem produtos')
  }
})
```

Esse teste passa com produtos e passa sem produtos. Ele não afirma nada — só descreve o que encontrou.

JAVASCRIPT

```javascript
// ✅ BOM — o teste CRIA a condição que quer validar, e aí afirma
cy.intercept('GET', '**/produtos*', { fixture: 'produtos-mock.json' }).as('produtos')

cy.loginViaApi()
cy.visit('/admin/listarprodutos')
cy.wait('@produtos')

cy.get('table.table-striped tbody tr').should('have.length', 2)
```

DICA

A regra por trás: se você não sabe o que esperar, o problema não é o `if` — é que falta preparar o estado. Teste bom controla a condição inicial.

Testes instáveis

Teste instável é aquele que passa às vezes. Causas, em ordem de frequência:

1. Espera por tempo em vez de por condição
2. Dependência de estado entre testes
3. Massa compartilhada sendo alterada por terceiros
4. Animação ou transição não aguardada
5. Requisição assíncrona não interceptada
6. Elemento que se redesenha entre a busca e a ação

Como diagnosticar:

```
TERMINAL
npx cypress run --spec "caminho/do/teste.cy.js"
```

Rode umas dez vezes.

- Falhou **1 em 10**? É instável.
- Falhou **sempre**? É bug — no teste ou na aplicação. Bem mais fácil de resolver.

A regra do teste instável

> **Conserte em até dois dias, ou tire da suíte principal. Não há meio termo.**

O motivo não é técnico, é humano. Teste instável ensina o time a ignorar o vermelho. E quando todo mundo aprende que o vermelho não significa nada, **a suíte inteira perde valor — inclusive os testes bons.**

Um teste instável é pior que teste nenhum.

Discussão (15 min): quais desses antipadrões você já viu? Quais já cometeu?

3.4 Cucumber (visão geral)

O que é BDD e o que é Gherkin

BDD (Behavior Driven Development) é uma prática de conversa: negócio, quem desenvolve e QA descrevem juntos o comportamento esperado, em linguagem natural, **antes** de existir código.

Gherkin é o formato de escrita dessa descrição, numa linguagem que todo o time lê. O **Cucumber** faz o Gherkin rodar como teste. Muitos times usam os três juntos.

```gherkin
# language: pt

Funcionalidade: Login na loja

  Cenário: Login com credenciais válidas
    Dado que estou na página de login
    Quando informo o e-mail "admin.treinamento@qa.com" e a senha "treino123"
    E clico no botão entrar
    Então devo visualizar a home do administrador
```

Os step definitions

Cada linha do cenário vira uma função:

```javascript
import { Dado, Quando, E, Entao } from '../../../support/cucumber-pt'
import LoginPage from '../../../pages/LoginPage'

Dado('que estou na página de login', () => {
  LoginPage.visitar()
})

Quando('informo o e-mail {string} e a senha {string}', (email, senha) => {
  LoginPage.preencherEmail(email).preencherSenha(senha)
})

E('clico no botão entrar', () => {
  LoginPage.clicarEntrar()
})

Entao('devo visualizar a home do administrador', () => {
  LoginPage.validarRedirecionamentoAdmin()
})
```

> **DICA**
>
> **Repare no que está acontecendo aqui.** Os steps chamam o **mesmo Page Object** do bloco anterior. Cucumber não é uma arquitetura paralela — é uma **camada de expressão** por cima do que você já tem. O motor continua sendo o mesmo Cypress.

ATENÇÃO

Por que o import vem de `cucumber-pt` e não do pacote? Até a versão 19 o `@badeball/cypress-cucumber-preprocessor` exportava `Dado`, `Quando` e `Entao` direto. Da versão 20 em diante, só sobraram `Given`, `When` e `Then`.

O projeto resolve isso com um arquivo de poucas linhas:

JAVASCRIPT

```
import { Given, When, Then } from '@badeball/cypress-cucumber-preprocessor'

export const Dado = Given
export const Quando = When
export const Entao = Then
export const E = When
```

O `.feature` continua 100% em português — disso cuida o `# language: pt`. Só o nome da função que registra o passo é que mudou.

Se você vir `(0 , import_cypress_cucumber_preprocessor.Dado) is not a function`, é isso. Leia só o fim: "*`Dado` não é uma função*". É um bom exemplo de como se lê um erro de biblioteca: a mensagem parece absurda, mas está dizendo a verdade.

O que faz o BDD render

- Negócio **lê e contribui** com as features
- O cenário nasce **antes** do código, no refinamento com negócio, desenvolvimento e QA
- Linguagem de **negócio**, que qualquer pessoa do time entende
- Features curtas, com steps reaproveitados e apoiados em Page Objects

Dois pontos de atenção, comuns em qualquer time e fáceis de ajustar: a feature escrita só pelo QA, depois do teste, e o cenário que descreve cliques em vez de comportamento.

DICA

O `login.feature` deste material usa passos de tela (`E clico no botão entrar`) de propósito, para mostrar o encaixe com o Page Object. Numa feature de negócio, o passo seria algo como `Quando entro como administrador`.

O valor está na conversa

O Gherkin registra o que negócio, desenvolvimento e QA combinaram. O Cucumber transforma esse registro numa especificação que **roda**. Quando a conversa acontece, o time ganha as duas coisas: acordo sobre o comportamento e um teste que prova esse acordo a cada execução.

E os fundamentos destes três dias — asserção, independência, massa criada pelo próprio teste — valem igual em Gherkin e em JavaScript. O formato, sozinho, não faz o teste.

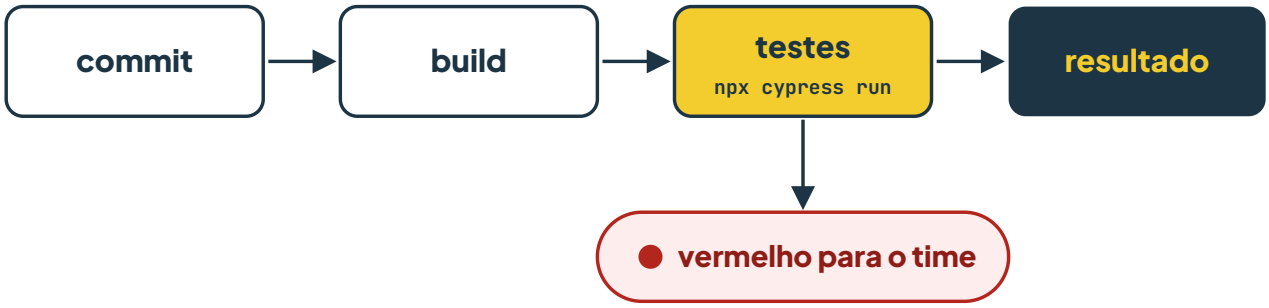
Como toda biblioteca, o Cucumber muda com o tempo: você acabou de ver uma mudança de versão trocar os nomes em português. Fixe a versão e leia as notas antes de atualizar.

3.5 CI/CD com Docker e Jenkins

Por que CI

> **O teste que só roda na sua máquina não protege ninguém.**

Integração contínua é um servidor que roda sua suíte automaticamente a cada mudança no código. Se quebrar, todo mundo fica sabendo na hora — não duas semanas depois.



DICA

A mensagem central deste bloco: o comando que roda no servidor é **exatamente** o `npx cypress run` que você já usa desde o Dia 1. Não existe mágica no CI. Existe outra máquina rodando o que você já roda.

Docker

Docker empacota a aplicação com tudo de que ela precisa. A imagem oficial do Cypress já vem com Node e navegadores instalados.

DOCKERFILE

```dockerfile
FROM cypress/included:15.21.1

WORKDIR /e2e

COPY package*.json ./
RUN npm ci

COPY . .

ENV CYPRESS_baseUrl=https://front.serverest.dev CYPRESS_apiUrl=https://serverest.dev

ENTRYPOINT ["sh", "-c"]
CMD ["node scripts/preparar-ambiente.js && npx cypress run"]
```

Três linhas pedem explicação:

- **`ENV`.** O `cypress.config.js` aponta para o `localhost`, onde roda a loja do treinamento. Dentro do container não existe loja nenhuma: no CI o alvo é o ServeRest público. Qualquer variável começando com `CYPRESS_` sobrescreve a configuração — a mesma troca de ambiente do Dia 1, feita pelo lado de fora
- **`ENTRYPOINT`.** A imagem do Cypress já vem com `ENTRYPOINT ["cypress", "run"]`, ou seja, ela roda os testes sozinha ao iniciar. Trocamos por um shell para poder rodar dois comandos em sequência
- **`preparar-ambiente` antes do `cypress run`.** O ServeRest público é zerado com frequência; o script garante o usuário admin. O comando de teste continua sendo exatamente o `npx cypress run` do Dia 1

> **ATENÇÃO**
> Junto do Dockerfile vai um `.dockerignore` com `node_modules`. Sem ele, o `COPY . .` leva o `node_modules` da sua máquina para dentro da imagem, por cima do que o `npm ci` instalou — e no Windows ou no Mac isso traz binários da plataforma errada.

> **CRÍTICO**
> **Fixe a versão. Nunca use `latest`.** Ambiente de CI que muda sozinho gera **falha fantasma** — o teste quebra e você perde meio dia procurando o que mudou no seu código, quando o que mudou foi a imagem. E falha fantasma destrói a confiança na suíte mais rápido que qualquer bug.

Construindo e rodando, a partir da raiz do projeto:

TERMINAL

```
docker build -f ci/Dockerfile -t testes-cypress .
docker run --rm testes-cypress
```

Aplicação e testes em containers separados (`ci/docker-compose.yml`, resumido):

YAML

```yaml
services:
  serverest:
    image: paulogoncalvesbh/serverest:3.2.2
    ports:
      - "3100:3000"

  cypress:
    build:
      context: ..
      dockerfile: ci/Dockerfile
    depends_on:
      - serverest
    environment:
      - CYPRESS_apiUrl=http://serverest:3000
      - CYPRESS_baseUrl=http://serverest:3000
    command:
      - >-
        node scripts/preparar-ambiente.js &&
        npx cypress run
        --spec "cypress/e2e/dia2/06-api-usuarios.cy.js,cypress/e2e/dia2/07-api-produtos.cy.js"
```

TERMINAL

```
docker compose -f ci/docker-compose.yml up --build --abort-on-container-exit
```

Um container com a API, outro com os testes, sem depender de internet.

ATENÇÃO

Por que só os specs de API? A imagem oficial do ServeRest é **somente API**. A loja original é outro projeto, com o endereço da API **fixo no código**, apontando para o ServeRest público. A loja do treinamento só roda localmente porque o `serverest-treinamento` traz um build dela com esse endereço trocado. Este compose não inclui a loja; então, no container, só os testes de API fazem sentido. É uma lição real: containerizar o ambiente depende de a aplicação permitir — ou de alguém fazer o trabalho de adaptá-la.

DICA

Repare no `CYPRESS_apiUrl` do compose. Ele vence o `ENV` do Dockerfile: a mesma imagem testa o ServeRest público ou o container ao lado, sem reconstruir nada.

Como é a execução

Os dois containers escrevem no mesmo terminal, e cada linha diz de qual veio. Este trecho é da execução real (a íntegra está em `ci/execucao-registrada.txt`):

```
cypress-1    | ✅ Pronto. admin.treinamento@qa.com / treino123 — administrador.
...
cypress-1    |   API de usuários — escrita
serverest-1  | POST /usuarios 201 3.334 ms - 82
cypress-1    |     ✓ deve cadastrar um novo usuário
serverest-1  | POST /usuarios 400 0.623 ms - 53
cypress-1    |     ✓ deve impedir cadastro com e-mail duplicado
...
cypress-1    |     ✓  All specs passed!                        824ms       11       11
cypress-1 exited with code 0
```

Três coisas para reparar:

- **Cada `✓` vem logo depois da requisição que ele fez.** Dá para ver o teste conversando com a API
- **O `400` é sucesso.** O teste de duplicidade *pediu* o erro, e recebeu
- **`exited with code 0`** é o único número que o servidor de CI lê. Zero é verde; qualquer outro, vermelho

Jenkins

JENKINSFILE

```
pipeline {
    agent {
        docker {
            image 'cypress/included:15.21.1'
            args '--entrypoint='
        }
    }

    environment {
        CYPRESS_senhaAdmin = credentials('serverest-senha-admin')
        CYPRESS_baseUrl = "${env.LOJA_URL ?: 'https://front.serverest.dev'}"
        CYPRESS_apiUrl = "${env.API_URL ?: 'https://serverest.dev'}"
        NO_COLOR = '1'
    }

    stages {
        stage('Instalar dependências') {
            steps { sh 'npm ci' }
        }

        stage('Preparar ambiente') {
            steps { sh 'node scripts/preparar-ambiente.js' }
        }

        stage('Executar testes') {
            steps { sh 'npx cypress run' }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'cypress/videos/**, cypress/screenshots/**',
                             allowEmptyArchive: true
            publishHTML target: [
                reportDir: 'cypress/reports',
                reportFiles: 'index.html',
                reportName: 'Relatório Cypress'
            ]
        }
    }
}
```

O `args '--entrypoint='` desliga o `cypress run` automático da imagem: o Jenkins precisa de um container parado, esperando os comandos de cada estágio.

As duas URLs usam `LOJA_URL` e `API_URL` se o Jenkins as definir, e o ServeRest público se não: a mesma troca de ambiente do Dia 1, feita pelo Jenkins. O Jenkins da aula aponta para a loja do instrutor.

O `NO_COLOR` desliga as cores do Cypress. O Jenkins roda o container com terminal, o Cypress manda cores, e o console do Jenkins não sabe exibi-las: sem essa linha, a saída vira `[90m—[39m` repetido

centenas de vezes.

Três detalhes que valem atenção:

`npm ci`, não `npm install`. O `ci` respeita exatamente o `package-lock.json`. O `install` pode atualizar versões — e aí você tem um ambiente que ninguém consegue reproduzir.

`credentials(...)`. A senha vem do cofre do Jenkins, não do código. Quem tem acesso ao repositório não tem acesso à senha.

`post { always { ... } }`, não `success`. As evidências são guardadas mesmo quando o teste falha — que é exatamente quando você mais precisa delas.

Como é a execução no Jenkins

Em aula, o instrutor cria o job pela tela do Jenkins e roda ao vivo: um build verde, um vermelho (com um teste quebrado de propósito) e o conserto. A receita desse Jenkins, para estudar na sua máquina, está em `ci/jenkins-local/` (`Dockerfile`, `plugins.txt`, `casc.yaml` e `docker-compose.yml`).

Este pipeline foi executado num Jenkins de verdade. Cada build vira uma linha, cada estágio uma coluna:



O build 1 ficou vermelho, e não foi culpa do Jenkins: um teste da própria suíte perdeu uma corrida no servidor, que é mais lento que a máquina local (a história está no 2.4, na caixa "Por que `should` e não `then`?"). O CI achou um teste instável que ninguém tinha visto. Depois da correção, dois builds verdes.

No console, o que aparece depois de cada `+` é o comando que o Jenkins digitou. São os mesmos de sempre: `npm ci`, `node scripts/preparar-ambiente.js`, `npx cypress run`. A saída completa está em `ci/jenkins-execucao-registrada.txt`.

ATENÇÃO

O relatório abre em branco no Jenkins? É a política de segurança do Jenkins (CSP), que bloqueia script e estilo em relatórios publicados, e o Mochawesome precisa dos dois. Quem administra o Jenkins ajusta a propriedade `hudson.model.DirectoryBrowserSupport.CSP`. A configuração usada neste material está em `ci/jenkins-local/docker-compose.yml`. Não desligue a proteção inteira: libere só o que o relatório precisa.

Boas práticas de CI

- Segredos como variável de ambiente do servidor, nunca no código
- Testes rápidos e independentes, para permitir execução em paralelo
- **A falha tem que quebrar o build.** CI que não quebra é enfeite
- Teste instável precisa ser tratado **antes** de entrar na esteira

---

3.6 Relatórios

Para quem serve

> **Relatório é para quem não estava lá.**

Se ele precisa de você ao lado para ser entendido, não está bom.

Mochawesome

TERMINAL

```
npm install --save-dev cypress-mochawesome-reporter
```

JAVASCRIPT

```javascript
module.exports = defineConfig({
  reporter: 'cypress-mochawesome-reporter',
  reporterOptions: {
    reportDir: 'cypress/reports',
    charts: true,
    reportPageTitle: 'Relatório de Testes — Treinamento Cypress',
    embeddedScreenshots: true,
    inlineAssets: true
  },
  e2e: {
    setupNodeEvents(on, config) {
      require('cypress-mochawesome-reporter/plugin')(on)
      return config
    }
  }
})
```

E no `cypress/support/e2e.js`:

JAVASCRIPT

```javascript
import 'cypress-mochawesome-reporter/register'
```

ATENÇÃO

Sem esse `import` no arquivo de suporte, o relatório é gerado mas sai sem os screenshots embutidos. É fácil esquecer.

TERMINAL

```
npx cypress run
# abra cypress/reports/index.html
```

Você ganha: gráfico de aprovados e reprovados, tempo por suíte, screenshot embutido nas falhas, e filtro por status.

O que mata um relatório

Nome de teste ruim.

```
❌ × teste 1
❌ × login
❌ × deve funcionar

✅ × deve exibir mensagem de erro ao informar senha inválida
```

Olhe a lista de cima. O que quebrou? Você não sabe — precisa abrir o código. Olhe a de baixo: você já sabe, e já sabe se é urgente.

> DICA
> **É o antipadrão 8 voltando.** Nome de teste não é detalhe de organização. É a interface da sua suíte com o resto do mundo.

Outras opções, que vale conhecer de nome: Allure Report, JUnit XML (para integrar com o Jenkins), Cypress Cloud.

3.7 Banco de dados com `cy.task`

Para que ir ao banco num teste

Três usos legítimos:

1. **Conferir o que foi gravado.** A tela disse "sucesso". O dado está no banco, do jeito certo?
2. **Preparar um estado** que nem a tela nem a API deixam criar: um registro antigo, um cadastro pela metade, um status que só um processo noturno põe
3. **Limpar** o que o teste criou

E quando não ir: se a API faz, use a API. Teste que escreve direto no banco fica preso ao formato das tabelas, e esse formato muda sem avisar quem testa. Nunca em produção. E o banco não substitui a tela: ele complementa.

O navegador não fala com banco

O teste roda **dentro do navegador**, e o navegador não abre conexão com banco (nem deve: seria entregar a senha do banco a qualquer página). Quem abre é o **Node**, que roda o Cypress por trás.

O `cy.task` é a ponte: o teste pede uma tarefa **pelo nome**; o Node executa a função com esse nome e devolve o resultado ao teste.

```
teste (navegador) ──cy.task('sql', ...)──▶ Node (setupNodeEvents) ──pg──▶ PostgreSQL
        ◀──────────────── as linhas voltam ────────────────
```

Se o teste chamar uma tarefa que o Node não conhece, o Cypress diz exatamente isso:

```
cy.task('sql') failed with the following error:
The task 'sql' was not handled in the setupNodeEvents method.
```

O que é uma task, em detalhe

Task (tarefa) é uma função que mora do lado do **Node** e que o teste chama pelo nome. Ela existe porque o teste roda dentro do navegador, e o navegador não pode fazer várias coisas: abrir conexão com banco,

ler um arquivo do disco, usar uma biblioteca do Node. A task faz por ele.

São sempre três peças, ligadas pelo nome:

```javascript
// 1. A função, no Node — cypress/tasks/banco.js
module.exports = {
  async sql({ comando, valores = [] }) {
    const { rows } = await pool.query(comando, valores)
    return rows
  }
}

// 2. O registro — cypress.config.js, dentro do setupNodeEvents
on('task', require('./cypress/tasks/banco'))

// 3. A chamada, no teste
cy.task('sql', { comando: 'SELECT current_database() AS banco' })
```

O `require` carrega o arquivo, e cada função dele vira uma task com o mesmo nome. Se uma das peças faltar, ou o nome não bater, o erro é `The task 'sql' was not handled in the setupNodeEvents method`.

O contrato — conferido regra por regra:

Regra	Na prática
A task recebe **um** argumento	Para mandar vários valores, use um objeto: `cy.task('sql', { comando, valores })`
Ela precisa **devolver alguma coisa**	Sem nada para devolver, `return null`. Sem retorno, o teste falha com `The task 'x' returned undefined. You must return a value, null, or a promise...`
Pode ser `async`	O Cypress espera a promessa terminar e entrega o resultado
O retorno viaja como **JSON**	Uma data chega ao teste como texto; uma função não chega
Erro dentro da task deixa o teste **vermelho**	Com a mensagem do erro: `connect ECONNREFUSED 127.0.0.1:5433`
`console.log` dentro da task aparece no **terminal**	Não no console do navegador: é outro programa
O Cypress espera até **60 segundos**	Para mais: `cy.task('sql', { ... }, { timeout: 120000 })`

Para que mais serve: tudo o que o navegador não pode fazer. Ler e gravar arquivos, usar bibliotecas do Node (gerar massa, ler uma planilha), preparar ou limpar o ambiente, escrever no terminal do CI. O banco é só um dos usos.

O banco da demonstração

Um PostgreSQL em Docker, com uma tabela só. Dois arquivos na pasta `banco/`:

YAML

```yaml
# banco/docker-compose.yml
services:
  postgres:
    image: postgres:17.6-alpine
    container_name: banco-treinamento
    environment:
      POSTGRES_DB: treinamento
      POSTGRES_USER: treinamento
      POSTGRES_PASSWORD: treino123
    ports:
      - "5433:5432"
    volumes:
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U treinamento -d treinamento"]
      interval: 2s
      timeout: 3s
      retries: 30
```

SQL

```sql
-- banco/init.sql — roda uma vez, quando o banco nasce
CREATE TABLE clientes (
  id        SERIAL PRIMARY KEY,
  nome      TEXT        NOT NULL,
  email     TEXT        NOT NULL UNIQUE,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

TERMINAL

```
docker compose -f banco/docker-compose.yml up -d --wait   # sobe
docker compose -f banco/docker-compose.yml down            # para e apaga os dados
```

> **DICA**
>
> **Por que a porta 5433, e não a 5432 padrão?** Muita máquina já tem um PostgreSQL instalado ocupando a 5432 — a de quem montou este material tinha. O `"5433:5432"` diz: a 5433 da sua máquina leva à 5432 de dentro do container.

E o driver do PostgreSQL para o Node:

TERMINAL

```
npm install --save-dev pg@8.23.0
```

A tarefa, do lado do Node

`cypress/tasks/banco.js` (o arquivo completo tem também a tarefa do banco da loja, que vem logo abaixo):

JAVASCRIPT

```
const { Pool } = require('pg')

// As credenciais vêm de variável de ambiente. O valor depois do || é o banco
// de estudo; no trabalho, só a variável.
const pool = new Pool({
  host: process.env.DB_HOST || '127.0.0.1',
  port: Number(process.env.DB_PORT || 5433),
  database: process.env.DB_NAME || 'treinamento',
  user: process.env.DB_USER || 'treinamento',
  password: process.env.DB_PASSWORD || 'treino123'
})

module.exports = {
  // O SQL e os valores vão separados: o pg encaixa cada valor no $1, $2...
  async sql({ comando, valores = [] }) {
    const { rows } = await pool.query(comando, valores)
    return rows
  }
}
```

E uma linha no `cypress.config.js`, dentro do `setupNodeEvents`, antes do `return config`:

JAVASCRIPT

```
on('task', require('./cypress/tasks/banco'))
```

> **DICA**
>
> **Por que `127.0.0.1` e não `localhost`?** Com o banco fora do ar e `localhost`, o erro é só `AggregateError`: o Node tenta os dois endereços da máquina (IPv6 e IPv4) e junta as duas falhas. Com `127.0.0.1`, o erro diz o que houve: `connect ECONNREFUSED 127.0.0.1:5433`.

Os testes

`cypress/e2e/dia3/banco/13-banco-de-dados.cy.js`:

JAVASCRIPT

```javascript
import CadastroUsuarioPage from '../../../pages/CadastroUsuarioPage'

describe('PostgreSQL — a conexão pelo cy.task', () => {
  it('deve consultar o banco de dados', () => {
    cy.task('sql', { comando: 'SELECT current_database() AS banco' })
      .its('0.banco')
      .should('eq', 'treinamento')
  })

  it('deve gravar e ler um cliente', () => {
    const email = `cliente_${Date.now()}@qa.com`

    cy.task('sql', {
      comando: 'INSERT INTO clientes (nome, email) VALUES ($1, $2) RETURNING id',
      valores: ['Cliente do Banco', email]
    }).then(([cliente]) => {
      cy.task('sql', {
        comando: 'SELECT nome, email FROM clientes WHERE id = $1',
        valores: [cliente.id]
      })
        .its(0)
        .should('deep.equal', { nome: 'Cliente do Banco', email })
    })
  })

  it('deve apagar o cliente criado pelo teste', () => {
    cy.task('sql', {
      comando: 'INSERT INTO clientes (nome, email) VALUES ($1, $2) RETURNING id',
      valores: ['Cliente a Apagar', `apagar_${Date.now()}@qa.com`]
    }).then(([cliente]) => {
      cy.task('sql', { comando: 'DELETE FROM clientes WHERE id = $1', valores: [cliente.id] })

      cy.task('sql', { comando: 'SELECT id FROM clientes WHERE id = $1', valores: [cliente.id] })
        .should('be.empty')
    })
  })
})
```

Três coisas para reparar:

- **O resultado é uma lista de linhas.** `.its('0.banco')` lê a coluna `banco` da primeira linha; `.then(([cliente]) => ...)` pega a primeira linha
- `RETURNING id` faz o `INSERT` devolver o que acabou de gravar. É assim que o teste sabe qual registro conferir e apagar
- **Massa única também no banco.** O `email` é `UNIQUE`, como no ServeRest. Com e-mail fixo, a segunda execução falharia com `duplicate key value violates unique constraint`

CRÍTICO

`$1`, **nunca** `${}`. O SQL e os valores vão **separados**, e o driver encaixa cada valor com segurança. Se você colar o valor dentro do texto (`` `... WHERE email = '${email}'` ``), um nome como `D'Ávila` quebra o SQL no apóstrofo — e, se o valor vier de alguém mal intencionado, ele escreve SQL no lugar do dado. Chama-se *SQL injection*, e o `$1` fecha essa porta.

O banco da própria loja

Até aqui o PostgreSQL estava sozinho, sem aplicação. A loja do treinamento tem o próprio banco: o ServeRest usa o **NeDB**, um arquivo de texto com um registro JSON por linha, dentro da instalação dele.

A segunda tarefa do `cypress/tasks/banco.js` lê esse arquivo:

JAVASCRIPT

```
const fs = require('fs')
const path = require('path')

// O banco da loja fica dentro da instalação do ServeRest, na pasta ao lado.
const DADOS_DA_LOJA =
  process.env.SERVEREST_DADOS ||
  path.resolve(__dirname, '../../../serverest-treinamento/node_modules/serverest/src/data')

// ...dentro do module.exports, ao lado da tarefa sql:
  buscarUsuarioNoBancoDaLoja(email) {
    const linhas = fs
      .readFileSync(path.join(DADOS_DA_LOJA, 'usuarios.db'), 'utf8')
      .split('\n')
      .filter(Boolean)

    // O NeDB nunca altera uma linha, só acrescenta: vale a última de cada _id.
    const usuarios = new Map()
    for (const linha of linhas) {
      const registro = JSON.parse(linha)
      if (!registro._id) continue
      if (registro.$$deleted) usuarios.delete(registro._id)
      else usuarios.set(registro._id, registro)
    }

    // Uma task não pode devolver undefined: o cy.task falharia. null pode.
    return [...usuarios.values()].find((usuario) => usuario.email === email) || null
  }
```

E o teste, no mesmo arquivo de spec:

JAVASCRIPT

```
describe('O banco da própria loja', () => {
  it('deve gravar no banco o usuário cadastrado pela interface', () => {
    const usuario = { nome: 'Cliente pela Tela', email: `tela_${Date.now()}@qa.com`, password:
'teste123' }

    CadastroUsuarioPage.cadastrar(usuario, { administrador: true })
      .validarMensagem('Cadastro realizado com sucesso')

    // A tela disse "sucesso". O banco diz o que foi gravado de fato.
    cy.task('buscarUsuarioNoBancoDaLoja', usuario.email).should('include', {
      nome: usuario.nome,
      administrador: 'true'
    })
  })
})
```

É o uso número 1 numa aplicação de verdade: a tela diz uma coisa, o banco confirma.

> **DICA**
>
> **Por que ler o arquivo, e não usar o driver do NeDB?** Ao abrir o arquivo, o driver o reescreve. Com a loja no ar usando o mesmo arquivo, os dois programas brigariam. Só ler é seguro.

As armadilhas do dia a dia

Conferidas uma a uma contra o PostgreSQL da demonstração:

Armadilha	O que acontece	O que fazer
Número e data como texto	`count(*)` chega como `"1"`, `numeric` como `"10.50"`, e data como texto em UTC (`"2026-09-29T00:11:02.132Z"`)	Compare com texto, ou converta no SQL: `count(*)::int` chega como `1`
O `should` não repete o `cy.task`	A consulta roda **uma vez**. Se a aplicação grava com atraso (fila, processo em segundo plano), o teste olha cedo demais e falha na hora: `expected [] to have a length of 1 but got 0`	Espere no Node, com a tarefa `esperarNoBanco` (abaixo)
Consulta lenta	O `cy.task` espera até 60 segundos	`cy.task('sql', { ... }, { timeout: 120000 })`
Valor colado no SQL	Com um apóstrofo (`D'Ávila`), o banco responde `syntax error at or near "Ávila"`	Sempre `$1`, `$2`
Massa fixa	`duplicate key value violates unique constraint`	Dado único por execução

Dado que chega atrasado

A espera fica do lado do Node, na tarefa, e não no teste: consulta de meio em meio segundo até achar alguma linha, ou falha com mensagem clara quando o prazo acaba. No teste não entra `cy.wait` nenhum.

JAVASCRIPT

```
// cypress/tasks/banco.js, dentro do module.exports
  async esperarNoBanco({ comando, valores = [], segundos = 10 }) {
    const limite = Date.now() + segundos * 1000
    for (;;) {
      const { rows } = await pool.query(comando, valores)
      if (rows.length) return rows
      if (Date.now() > limite) throw new Error(`Nada no banco depois de ${segundos} s: ${comando}`)
      await new Promise((ok) => setTimeout(ok, 500))
    }
  },
```

JAVASCRIPT

```
cy.task('esperarNoBanco', {
  comando: 'SELECT nome FROM clientes WHERE email = $1',
  valores: [email]
})
  .its('0.nome')
  .should('eq', 'Cliente do Banco')
```

Conferido com um registro gravado 2 segundos depois de o teste começar a esperar: achou em 2,1 s. Sem o registro, a falha diz `Nada no banco depois de 10 s: SELECT ...`.

Para conectar no banco da sua empresa

- Peça ao DBA: endereço, porta, nome do banco e um **usuário de teste** com permissão só no que o teste precisa
- Confira o **acesso de rede**: da sua máquina e do servidor de CI até o banco (VPN, firewall)
- Senha por **variável de ambiente**. No PowerShell, antes de abrir o Cypress:

  POWERSHELL

  ```
  $env:DB_HOST="endereco-do-banco"
  $env:DB_PASSWORD="a-senha"
  npx cypress open
  ```

  No CI, do cofre, como o `credentials()` do Jenkinsfile
- Se o banco exigir conexão segura, o `Pool` aceita a opção `ssl`: o DBA diz qual configuração usar
- **Banco de teste, nunca produção**
- Uma tarefa com nome de negócio para cada necessidade, em vez de SQL espalhado pelos testes

Muda o banco, muda o driver

O padrão é o mesmo: **uma tarefa no Node, um `cy.task` no teste.**

Banco	Pacote do Node
PostgreSQL	`pg`
SQL Server	`mssql`
MySQL	`mysql2`
Oracle	`oracledb`

A tarefa muda por dentro; o teste não muda nada.

Boas práticas com banco

- Senha por **variável de ambiente**; no CI, do cofre (como o `credentials()` do Jenkinsfile)
- Sempre `$1`, nunca `${}` no SQL
- **Banco de teste**, nunca produção
- Uma tarefa com **nome de negócio** (`buscarUsuarioNoBancoDaLoja`) em vez de SQL espalhado pelos testes — é o Page Object do banco
- Se precisar limpar, limpe **antes** do teste: se ele falhar no meio, um "depois" não roda
- Se a API faz, use a API

3.8 Lab final, code review e encerramento

Lab final

Crie o arquivo `cypress/e2e/dia3/lab-final.cy.js`.

Parte 1 — Preparação via API

- Criar um usuário administrador
- Autenticar e obter o token
- Cadastrar um produto

Parte 2 — Interface

- Entrar usando login via API
- Validar o produto na listagem
- Excluir o produto pela interface e validar que ele sumiu

Parte 3 — Data driven

- Fixture com três produtos
- Cadastrar todos em loop
- Validar a quantidade

Parte 4 — Resiliência

- Interceptar a listagem e simular erro 500
- Validar o comportamento da tela

Requisitos técnicos:

- Ao menos um Page Object
- Ao menos dois comandos customizados
- Ao menos um alias de requisição
- Nenhum `cy.wait` numérico
- Nenhum antipadrão do catálogo do bloco 3.3
- Relatório Mochawesome gerado

Critério de pronto

> `npx cypress run` **verde do zero, duas vezes seguidas.**

Rodar duas vezes expõe dependência de estado. É o teste do teste.

> ATENÇÃO
> **No ServeRest isso tem uma consequência prática:** nome de produto é único. Se você fixar o nome na fixture, a primeira rodada passa e a segunda falha com `Já existe produto com esse nome`. A fixture guarda o **modelo**; o teste acrescenta um sufixo por execução.

> ATENÇÃO
> **Cuidado na Parte 3.** É tentador escrever um último teste que conta os produtos cadastrados pelos testes anteriores. Mas ele só passa se os outros rodarem antes: é o antipadrão 2. Cada teste deve cadastrar **e** validar o próprio produto.

Dois avisos sobre este lab

Sobre a Parte 2: excluir um produto pela tela **não gera mensagem de confirmação**. A página recarrega e o produto some. Sua asserção tem que ser a ausência dele, não um texto de sucesso.

E como as linhas da tabela não têm `data-testid`, a exclusão se ancora no nome:

```javascript
cy.contains('table.table-striped tbody tr', nomeDoProduto)
  .contains('button', 'Excluir')
  .click()
```

Sobre a Parte 4: a tela não trata o erro 500. **Isso não é um defeito do exercício — é o exercício.** Há duas respostas defensáveis: **afirmar que a exceção acontece**, como no bloco 2.4, ou **deixar o teste vermelho de propósito**, esperando a mensagem de erro que a tela deveria mostrar, e registrar o bug. A indefensável é esconder com um `return false` genérico.

A resolução mostrada em aula

Tente fazer antes de ler.

JAVASCRIPT

```javascript
import ProdutosPage from '../../pages/ProdutosPage'

const produtosFixture = require('../../fixtures/produtos.json')

describe('Lab Final — preparação e interface', () => {
  let admin

  // Parte 1: o próprio teste cria o administrador e obtém o token dele.
  beforeEach(() => {
    cy.criarUsuarioViaApi({ nome: 'Admin do Lab', administrador: 'true' }).then((usuario) => {
      admin = usuario
      cy.obterToken(admin.email, admin.password).should('contain', 'Bearer')
      cy.loginViaApi(admin.email, admin.password)
    })
  })

  it('deve exibir na tela o produto criado via API', () => {
    cy.intercept('GET', '**/produtos*').as('listagem')

    const dados = { nome: `Lab Final ${Date.now()}`, email: admin.email, senha: admin.password }
    cy.cadastrarProdutoViaApi(dados).then((produto) => {
      ProdutosPage.visitarListagem()

      cy.wait('@listagem').its('response.statusCode').should('eq', 200)
      ProdutosPage.validarProdutoNaListagem(produto.nome)
    })
  })

  it('deve remover o produto da listagem ao excluí-lo pela interface', () => {
    const dados = { nome: `Lab Final Excluir ${Date.now()}`, email: admin.email, senha: admin.password }
    cy.cadastrarProdutoViaApi(dados).then((produto) => {
      ProdutosPage.visitarListagem()
        .validarProdutoNaListagem(produto.nome)
        .excluirProduto(produto.nome)
        .validarProdutoForaDaListagem(produto.nome)
    })
  })
})

describe('Lab Final — data driven', () => {
  // O ServeRest recusa nome repetido: o sufixo deixa rodar duas vezes seguidas.
  const sufixo = `${Date.now()}`

  // Cada teste cadastra E valida o próprio produto. Um último teste que
  // contasse os anteriores dependeria da ordem: o antipadrão 2.
  produtosFixture.forEach((produto) => {
    it(`deve cadastrar "${produto.nome}" via API`, () => {
      cy.cadastrarProdutoViaApi({ ...produto, nome: `${produto.nome} ${sufixo}` }).then((criado) => {
        cy.request('GET', `${Cypress.env('apiUrl')}/produtos/${criado._id}`)
          .its('body')
          .should('include', { preco: produto.preco, quantidade: produto.quantidade })
      })
    })
```

```
  })
})

describe('Lab Final — resiliência', () => {
  it('deve evidenciar que a listagem não trata erro 500', () => {
    cy.intercept('GET', '**/produtos*', { statusCode: 500, body: {} }).as('erro500')

    // Captura a exceção para AFIRMAR que ela acontece, em vez de escondê-la.
    const erros = []
    cy.on('uncaught:exception', (err) => {
      erros.push(err.message)
      return false
    })

    cy.loginViaApi()
    ProdutosPage.visitarListagem()
    cy.wait('@erro500').its('response.statusCode').should('eq', 500)

    cy.wrap(erros).should((lista) => {
      expect(lista.join(' '), 'a listagem quebra em vez de avisar o usuário').to.contain('undefined')
    })
  })
})
```

Code review

> **DICA**
>
> **Por que revisar código alheio consolida mais que escrever o seu.** Revisando, você precisa **articular** por que algo está bom ou ruim. E articular é o que transforma regra decorada em julgamento próprio.

O checklist:

- [ ] Os nomes dos testes descrevem o cenário **e** o resultado esperado?
- [ ] Cada teste roda isoladamente?
- [ ] Todo teste tem ao menos uma asserção significativa?
- [ ] Os seletores estão na camada mais alta disponível (1 ou 2 sempre que existir)?
- [ ] Existe algum `cy.wait` com número?
- [ ] A massa é criada pelo próprio teste?
- [ ] O setup usa o caminho mais rápido (API em vez de tela)?
- [ ] Há repetição que deveria ser comando ou Page Object?
- [ ] Existe `if` decidindo o que validar?
- [ ] Há segredo ou credencial **real** no código? (a do admin de estudo do ServeRest é exceção — bloco 2.5)

O arquivo revisado em aula (`cypress/e2e/dia3/codigo-para-revisar.js`). Ele está errado de propósito. Ache pelo menos dez problemas antes de ler o gabarito.

JAVASCRIPT

```javascript
describe('Produtos', () => {
  it('teste 1', () => {
    cy.visit('/login')
    cy.get('.form-control').eq(0).type('admin.treinamento@qa.com')
    cy.get('.form-control').eq(1).type('treino123')
    cy.get('.btn-primary').click()
    cy.wait(3000)

    cy.visit('/admin/cadastrarprodutos')
    cy.get('[data-testid="nome"]').type('Mouse Gamer')
    cy.get('[data-testid="preco"]').type('150')
    cy.get('[data-testid="descricao"]').type('Mouse com RGB')
    cy.get('[data-testid="quantity"]').type('10')
    cy.get('[data-testid="cadastarProdutos"]').click()
  })

  it('teste 2', () => {
    cy.visit('/login')
    cy.get('.form-control').eq(0).type('admin.treinamento@qa.com')
    cy.get('.form-control').eq(1).type('treino123')
    cy.get('.btn-primary').click()
    cy.wait(3000)

    cy.visit('/admin/listarprodutos')
    cy.get('body').then(($body) => {
      if ($body.text().includes('Mouse Gamer')) {
        cy.log('produto cadastrado')
      }
    })
  })

  it('teste 3', () => {
    cy.request({
      method: 'DELETE',
      url: 'http://localhost:3000/produtos/k9Xq3LmP0aZ7wR2t',
      headers: { authorization: 'Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6Im1hcmlhQGVtcHJlc2EuY29tIn0' }
    })
  })

  // it('deve editar o produto', () => {
  //   falhando desde a semana passada, depois eu vejo
  // })
})
```

Gabarito:

No arquivo	Problema	Onde apareceu
`teste 1`, `teste 2`, `teste 3`	Nome vago (antipadrão 8)	3.3, 3.6
`.form-control` com `.eq(0)`, `.btn-primary`	Seletor por classe e posição, e existe `data-testid` (antipadrão 3)	1.7
`cy.wait(3000)`	Espera por tempo (antipadrão 1)	1.5, 2.2
Login pela tela no começo de cada teste	Setup lento (antipadrão 5)	2.4
O mesmo login copiado nos dois testes	Repetição: pede comando ou Page Object	2.6, 3.2
`'Mouse Gamer'` fixo	Massa fixa: a segunda execução falha com `Já existe produto com esse nome` (antipadrão 4)	2.5
Teste 1 cadastra e não confere nada	Sem asserção (antipadrão 6)	3.3
Teste 2 só passa se o teste 1 rodar antes	Dependência de ordem (antipadrão 2)	1.6
`if` no teste 2	Nunca falha (antipadrão 9)	3.3
Teste 3: token de uma pessoa no código	Segredo real — e expira em 10 minutos	2.5
Teste 3: id e URL da API fixos	Massa fixa; ignora o `apiUrl` do config	2.3, 2.5
Teste 3: `DELETE` sem asserção	Com token válido, excluir um id que não existe responde 200 (`Nenhum registro excluído`): o teste passa sem excluir nada	2.3
Teste comentado "depois eu vejo"	Dívida silenciosa (antipadrão 10)	3.3

> **DICA**
>
> **Numa revisão de verdade, se você não achar nada, a revisão foi superficial.** Rode a suíte duas vezes seguidas — o segundo `run` costuma revelar o que a leitura não revelou.

As oito regras

O treinamento inteiro cabe em oito frases.

1. Teste independente. Sem ordem, sem estado compartilhado.

2. Seletor dedicado a teste. Negocie o `data-testid` com o time.

3. Espere acontecimento, nunca tempo.

4. Massa criada pelo próprio teste.

5. Setup pela via mais rápida. API em vez de tela.

6. Nome de teste que explica o cenário.

7. Não automatize tudo. Automatize o que dói se quebrar.

8. Teste instável se conserta ou se remove. Não há meio termo.

Para onde ir depois

Você tem, agora, uma base sólida. O que vem a seguir, mais ou menos em ordem de retorno:

Assunto	Por que vale
Component Testing	Testar componentes isolados, muito mais rápido que E2E
Acessibilidade com `cypress-axe`	Cobertura que quase ninguém tem e que importa de verdade
Testes visuais	Pegar regressão de layout, que o E2E não pega
Cypress Cloud e paralelização	Quando a suíte passar de dez minutos
TypeScript em testes	Autocompletar nos seus comandos customizados
Real World App	Aplicação de referência do próprio Cypress, com suíte completa para estudar

O conselho que vale mais que a lista

> **Escreva testes para algo que você usa.** Uma suíte pequena para um sistema real do seu trabalho ensina mais em duas semanas que qualquer curso.
>
> E quando o teste falhar — e ele vai falhar — volte ao método de quatro passos do bloco 1.7. Era a ferramenta mais útil do primeiro dia e continua sendo.

APÊNDICES

Consulta rápida

A parte da apostila que você vai usar depois do treinamento.

A Comandos do Cypress por finalidade

Navegação

JAVASCRIPT

```
cy.visit('/login')          // abre uma página (relativa à baseUrl)
cy.reload()                 // recarrega
cy.go('back')               // volta
cy.go('forward')            // avança
cy.url()                    // a URL atual
cy.title()                  // o título da aba
```

Encontrar elementos

JAVASCRIPT

```
cy.get('[data-testid="email"]')     // por seletor CSS
cy.contains('Entrar')               // por texto
cy.contains('button', 'Entrar')     // por texto, dentro de um seletor
cy.get('form').find('input')        // dentro do elemento anterior
cy.get('li').first()                // primeiro
cy.get('li').last()                 // último
cy.get('li').eq(2)                  // terceiro (conta do zero)
cy.get('td').parent()               // o pai
cy.get('td').parents('tr')          // o ancestral que casar com o seletor
cy.get('ul').children()             // os filhos diretos
```

Interagir

```javascript
.click()
.click({ force: true })       // força o clique — use com muita parcimônia
.dblclick()
.type('texto')
.type('texto', { log: false })      // não mostra o valor no log (senhas)
.type('{enter}')              // teclas especiais entre chaves
.clear()
.check()                      // checkbox ou radio
.uncheck()
.select('Opção')              // <select>
.trigger('mouseover')
.scrollIntoView()
```

Trabalhar com valores

```javascript
.invoke('val')                // chama um método do elemento
.invoke('text')
.its('response.statusCode')   // acessa uma propriedade
.then((valor) => { })         // recebe o valor no callback
cy.wrap(objeto)               // transforma um valor comum em algo "cypressável"
```

Rede

```javascript
cy.request('GET', url)                          // requisição direta, fora da tela
cy.intercept('GET', '**/rota*').as('apelido')     // observa
cy.intercept('GET', '**/rota*', { fixture: 'x.json' })   // simula com arquivo
cy.intercept('GET', '**/rota*', { statusCode: 500 })     // simula erro
cy.wait('@apelido')                             // espera acontecer
```

Arquivos e dados

```javascript
cy.fixture('usuarios')               // lê um arquivo de cypress/fixtures
cy.fixture('usuarios').as('usuarios') // e guarda com apelido
```

Banco de dados

```javascript
cy.task('sql', { comando: 'SELECT ... WHERE id = $1', valores: [id] })  // pede ao Node
// a tarefa mora em cypress/tasks/banco.js, registrada no setupNodeEvents:
on('task', require('./cypress/tasks/banco'))
```

Debug

JAVASCRIPT

```
cy.log('mensagem')
cy.pause()
.debug()
cy.screenshot('nome')
```

Linha de comando

TERMINAL

```
npm run cy:open                                  # escrever testes
npm run cy:run                                   # rodar tudo
npx cypress run --spec "caminho/do/teste.cy.js"  # rodar um só
npx cypress run --headed                         # ver o navegador
npx cypress run --browser chrome                 # escolher o navegador
npx cypress run --env chave=valor                # passar variável
```

B Tabela de asserções

Existência e visibilidade

JAVASCRIPT

```
.should('exist')
.should('not.exist')
.should('be.visible')
.should('not.be.visible')
```

Conteúdo

JAVASCRIPT

```
.should('have.text', 'exatamente este texto')
.should('contain', 'parte do texto')
.should('not.contain', 'este texto não')
.should('have.value', 'valor do campo')
.should('be.empty')
```

Quantidade

JAVASCRIPT

```
.should('have.length', 3)
.should('have.length.greaterThan', 0)
.should('have.length.at.least', 1)
```

Estado

```javascript
.should('be.checked')
.should('not.be.checked')
.should('be.disabled')
.should('be.enabled')
.should('have.focus')
```

Atributos e classes

```javascript
.should('have.class', 'ativo')
.should('not.have.class', 'erro')
.should('have.attr', 'href', '/login')
.should('have.css', 'display', 'none')
```

Encadeando

```javascript
cy.get('[data-testid="entrar"]')
  .should('be.visible')
  .and('contain', 'Entrar')      // .and é um .should encadeado
  .and('not.be.disabled')
```

`expect` — dentro de um `.then()`

```javascript
cy.request('GET', url).then((res) => {
  expect(res.status).to.eq(200)
  expect(res.body).to.have.property('quantidade')
  expect(res.body.usuarios).to.be.an('array')
  expect(res.body.message).to.contain('sucesso')
  expect(res.body._id).to.not.be.empty
  expect(res.body).to.deep.include({ nome: 'Fulano' })
})
```

> **DICA**
>
> **`should` ou `expect`?** `should` tenta de novo até passar — use na tela. `expect` roda uma vez só — use dentro de `.then()`, quando o valor já chegou.

C Como ler uma falha: o método de 4 passos

1. Leia a primeira linha do erro. Ela diz o que o Cypress esperava e o que encontrou. Não pule para o código.

2. Olhe o screenshot da falha. A tela estava como você imaginava? Tinha um modal? Um carregando? A página abriu?

3. Volte um comando no Time Travel. Passe o mouse sobre o comando anterior. O estado antes da falha já estava errado? Se sim, o problema não é onde você achou.

4. Confira o seletor no DevTools.

```javascript
document.querySelectorAll('[data-testid="email"]')
```

Vazio? O seletor está errado ou o elemento ainda não existe.

D Mensagens de erro e o que significam

Mensagem	O que está acontecendo	O que fazer
`Timed out retrying... Expected to find element`	O seletor não encontrou nada no tempo	Confira o seletor no DevTools; veja se o elemento depende de uma requisição
`element is detached from the DOM`	A página se redesenhou entre achar e agir	Refaça o `cy.get()` mais perto da ação, ou espere a requisição terminar antes
`cy.click() failed because this element is being covered`	Tem algo por cima	Feche o modal/overlay. `{ force: true }` é último recurso, e esconde o problema
`cy.visit() failed trying to load`	URL errada ou aplicação fora do ar	Abra a URL no navegador. Se não abrir, rode `npm start` no `serverest-treinamento`
`Cypress.env() was removed in Cypress version 16.0.0`	O projeto está no Cypress 16, e o material é do 15	`npm install cypress@15.21.1 --save-dev`, ou migre para `Cypress.expose()` / `cy.env()` (bloco 2.5)
`Cypress could not verify that this server is running`	O `cypress run` conferiu o `baseUrl` antes de começar e não achou a aplicação	Mesma coisa: suba a loja (`npm start`) e rode de novo
`Cannot read properties of undefined`	Acesso a algo que não existe	Geralmente um `.then()` recebendo resposta vazia, ou fixture com estrutura diferente
`cy.type() can only accept a string or number`	Você passou `undefined` — uma variável que ninguém preencheu	Confira de onde vem o valor: fixture com a chave errada, ou variável que outro teste deveria ter preenchido (bloco 1.6)
Aviso `allowCypressEnv` no terminal	`Cypress.env()` depreciado no Cypress 15	Nada a fazer por ora; substitutos: `Cypress.expose()` e `cy.env()` (bloco 2.5)
`bad option: --smoke-test`	`ELECTRON_RUN_AS_NODE` no terminal	`unset ELECTRON_RUN_AS_NODE` (bash) ou `$env:ELECTRON_RUN_AS_NODE=$null` (PowerShell)

Dados, API e rede (Dia 2)

Mensagem	O que está acontecendo	O que fazer
`cy.type() cannot accept an empty string`	Você passou `''`	Não digite nada: `if (valor) cy.get(...).type(valor)` — como no bloco 2.5
`Cannot call cy.fixture() outside a running test`	Comando do Cypress fora de um `it` ou hook	Para gerar testes em loop, use `require` em vez de `cy.fixture()`
`cy.wait() timed out waiting for the 1st request`	O intercept não casou com a rota	Foi registrado depois do `visit`? O padrão casa com a URL real (confira na aba Network)?

Cucumber (Dia 3)

Mensagem	O que está acontecendo	O que fazer
`(0 , import_...Dado) is not a function`	Import dos steps do Cucumber vindo do pacote	Importe de `support/cucumber-pt.js`
`Step implementation missing`	O preprocessor não achou os step definitions	Configure `stepDefinitions` no `package.json`

Banco de dados (Dia 3)

Mensagem	O que está acontecendo	O que fazer
`The task 'sql' was not handled in the setupNodeEvents method`	O teste chamou uma tarefa que o Node não conhece	Registre: `on('task', require('./cypress/tasks/banco'))` no `setupNodeEvents`, e confira o nome
`connect ECONNREFUSED 127.0.0.1:5433`	O banco não está no ar, ou a porta é outra	Suba o banco (`docker compose ... up -d`) e confira a porta
`AggregateError` sozinho	O mesmo `ECONNREFUSED`, com `localhost`: o Node tentou IPv6 e IPv4	Use `127.0.0.1` no `host` da conexão, e a mensagem fica legível
`duplicate key value violates unique constraint`	Massa fixa no banco: o registro já existe	Dado único por execução (`Date.now()`), como na loja

---

E Seletores do ServeRest

> Conferidos contra a aplicação real. Se algo aqui não funcionar, a aplicação mudou — ela é de estudo e muda sem avisar.

`/login`

Elemento	Seletor
E-mail	`[data-testid="email"]`
Senha	`[data-testid="senha"]`
Entrar	`[data-testid="entrar"]`
Link cadastrar	`[data-testid="cadastrar"]`
Alerta	`.alert`

`/cadastrarusuarios`

Elemento	Seletor
Nome	`[data-testid="nome"]`
E-mail	`[data-testid="email"]`
Senha	`[data-testid="password"]` ⚠️
Checkbox admin	`[data-testid="checkbox"]`
Cadastrar	`[data-testid="cadastrar"]`

`/admin/home`

Elemento	Seletor
Cadastrar usuários	`[data-testid="cadastrarUsuarios"]`
Cadastrar produtos	`[data-testid="cadastrarProdutos"]`
Listar produtos	`[data-testid="listarProdutos"]`
Listar usuários	`[data-testid="listarUsuarios"]`
Logout	`[data-testid="logout"]`

`/admin/cadastrarprodutos`

Elemento	Seletor
Nome	`[data-testid="nome"]`
Preço	`[data-testid="preco"]`
Descrição	`[data-testid="descricao"]`
Quantidade	`[data-testid="quantity"]` ⚠️ em inglês
Imagem	`[data-testid="imagem"]`
Cadastrar	`[data-testid="cadastarProdutos"]` ⚠️ typo da aplicação

Listagens do admin

Não têm `data-testid`. Ancore no texto:

```javascript
cy.contains('table.table-striped tbody tr', 'Nome do Produto')
  .contains('button', 'Excluir')
  .click()
```

Títulos das telas

Rota	`<h1>`
`/admin/home`	`Bem Vindo <nome>`
`/home` (loja)	`Serverest Store`
`/admin/listarprodutos`	`Lista dos Produtos`
`/admin/listarusuarios`	`Lista dos usuários`

As três armadilhas

1. `senha` **é só do login.** No cadastro é `password`.
2. **O botão de cadastrar produto tem typo:** `cadastarProdutos`, sem o segundo "r".
3. `Serverest Store` **não é a home do admin.** É a loja do usuário comum.

F Comandos customizados do projeto

Interface

JAVASCRIPT

```
cy.login(email, senha)
// Login pela tela. Existe para ser TESTADO, não para ser usado como setup.

cy.loginViaApi(email, senha)
// Login pela porta dos fundos. É o setup padrão.
// Sem argumentos, usa admin.treinamento@qa.com / treino123.

cy.get('h1').validarTexto('Bem Vindo')
// Comando filho: valida o texto do elemento anterior.

cy.emailUnico('prefixo')
// Gera um e-mail único.
```

API

JAVASCRIPT

```
cy.obterToken(email, senha)
// Devolve o token no formato do header. Sem argumentos, usa o admin padrão.

cy.criarUsuarioViaApi({ nome, email, password, administrador })
// Todos os campos são opcionais. Devolve o usuário criado, com _id.

cy.cadastrarProdutoViaApi({ nome, preco, descricao, quantidade })
// Resolve a autenticação por dentro. Devolve o produto criado, com _id.

cy.excluirProdutoViaApi(id)
cy.excluirUsuarioViaApi(id)
```

Exemplos de uso

```javascript
// Criar um usuário e entrar com ele
cy.criarUsuarioViaApi().then((usuario) => {
  cy.loginViaApi(usuario.email, usuario.password)
  cy.visit('/admin/home')
  cy.contains('h1', usuario.nome).should('be.visible')
})

// Preparar um produto e validar na tela
cy.loginViaApi()
cy.cadastrarProdutoViaApi({ nome: `Produto ${Date.now()}` }).then((produto) => {
  cy.visit('/admin/listarprodutos')
  cy.contains('td', produto.nome).should('be.visible')
})
```

G Endpoints da API do ServeRest

Base: `http://localhost:3000` (a sua cópia, do `serverest-treinamento`). A pública, com as mesmas rotas, é `https://serverest.dev`.

Verbo	Rota	Autenticação	Uso
POST	`/login`	—	Autenticar, obter token
GET	`/usuarios`	—	Listar (aceita `?email=`, `?nome=`)
GET	`/usuarios/{id}`	—	Buscar um
POST	`/usuarios`	—	Criar
DELETE	`/usuarios/{id}`	—	Excluir
GET	`/produtos`	—	Listar
GET	`/produtos/{id}`	—	Buscar um
POST	`/produtos`	✅ Token	Criar
DELETE	`/produtos/{id}`	✅ Token	Excluir
POST	`/carrinhos`	✅ Token	Criar carrinho

Respostas que você vai encontrar

Situação	Status	Corpo
Login OK	200	`{ message: "Login realizado com sucesso", authorization: "Bearer ..." }`
Login inválido	401	`{ message: "Email e/ou senha inválidos" }`
Login sem o campo e-mail	400	`{ email: "email é obrigatório" }`
Cadastro OK	201	`{ message: "Cadastro realizado com sucesso", _id: "..." }`
E-mail duplicado	400	`{ message: "Este email já está sendo usado" }`
Produto sem token	401	`{ message: "Token de acesso ausente, inválido..." }`
Produto com nome repetido	400	`{ message: "Já existe produto com esse nome" }`
Exclusão OK	200	`{ message: "Registro excluído com sucesso" }`

Autenticação

JAVASCRIPT

```javascript
cy.request('POST', `${apiUrl}/login`, {
  email: 'admin.treinamento@qa.com',
  password: 'treino123'
}).then((res) => {
  const token = res.body.authorization   // já vem com "Bearer "

  cy.request({
    method: 'POST',
    url: `${apiUrl}/produtos`,
    headers: { authorization: token },
    body: { nome: 'x', preco: 1, descricao: 'y', quantidade: 1 }
  })
})
```

`localStorage` do front

Chave	Para quê
`serverest/userToken`	O que o guarda de rota confere
`serverest/userNome`	O que a home do admin imprime
`serverest/userEmail`	Impede excluir o próprio usuário

CRÍTICO

Faça um `cy.visit` **antes** de escrever no `localStorage`. É ele que estabelece a origem.

H As oito regras

1. Teste independente. Sem ordem, sem estado compartilhado.

2. Seletor dedicado a teste. Negocie o `data-testid` com o time.

3. Espere acontecimento, nunca tempo.

4. Massa criada pelo próprio teste.

5. Setup pela via mais rápida. API em vez de tela.

6. Nome de teste que explica o cenário.

7. Não automatize tudo. Automatize o que dói se quebrar.

8. Teste instável se conserta ou se remove. Não há meio termo.

Catálogo de antipadrões

#	Antipadrão	Por que quebra
1	`cy.wait(3000)`	Lento quando funciona, instável quando não
2	Teste dependente de ordem	Quebra em paralelo e no CI
3	Seletor por classe CSS ou caminho estrutural	Quebra na primeira mudança de layout
4	Massa fixa em ambiente compartilhado	Quebra quando outra pessoa usa o mesmo dado
5	Login pela tela em todo teste	Multiplica tempo e pontos de falha
6	Teste sem asserção	Passa sempre, não protege nada
7	Um teste gigante com trinta passos	Quando falha, não se sabe onde
8	Nome vago	Relatório inútil
9	`if` decidindo o que validar	Teste que se adapta não valida
10	Comentar o teste que falha	Dívida silenciosa

I Para onde ir depois

Documentação

- docs.cypress.io — documentação oficial, muito boa
- Cypress Real World App — aplicação de referência com suíte completa para estudar
- ServeRest — para continuar praticando. A sua cópia local (serverest-treinamento) continua funcionando depois do curso

Próximos assuntos

Assunto	Por que vale
Component Testing	Testar componentes isolados, muito mais rápido
`cypress-axe`	Acessibilidade — cobertura que quase ninguém tem
Testes visuais	Regressão de layout, que o E2E não pega
Cypress Cloud	Quando a suíte passar de dez minutos
TypeScript	Autocompletar nos seus comandos customizados

Comandos que você vai usar sempre

Estão no apêndice A, em "Linha de comando".

E o mais importante

Escreva testes para algo que você usa. Uma suíte pequena para um sistema real do seu trabalho ensina mais em duas semanas do que qualquer curso.

Quando o teste falhar — e ele vai falhar — volte ao **método de quatro passos** do apêndice C. É a ferramenta mais útil deste material.